



INTERNSHIP PROJECT DOCUMENTATION

Islamabad AQI Forecast System

An End-to-End Machine Learning Pipeline for Multi-Horizon Air Quality Prediction

Prepared as part of the Internship Program at 10Pearls

Intern Name: Syed Jaffar Raza Kazmi

Department / Track: Data Science

Date: September 1, 2026

Table of Contents

1. Introduction	1
2. Objectives	1
3. System Architecture	2
3.1 Component Overview	2
4. Data Pipeline	3
4.1 Data Sources	3
4.2 Live Ingestion and Station Fallback	3
4.3 Feature Engineering and Schema Enforcement	3
4.4 Historical Backfill	4
4.5 Feature Store	4
5. Machine Learning Model	5
5.1 Problem Formulation	5
5.2 Models Evaluated	5
5.3 Training Methodology	6
5.4 Results	6
6. Automation and Continuous Operation	6
7. Prediction Serving and Explainability	7
7.1 Live Inference	7
7.2 Model Explainability with SHAP	7
7.3 Hazard Alerting	8
8. Interactive Dashboard	8
8.1 Visibility of System Status	8
8.2 Recognition Rather Than Recall	8
8.3 Consistency and Standards	8
8.4 Error Prevention and Recovery	8
8.5 Aesthetic and Minimalist Design; Progressive Disclosure	9
8.6 Help and Documentation	9
8.7 Accessibility Considerations	9
9. Deployment	9
10. Challenges and Solutions	10
10.1 Sensor and Data Source Reliability	10
10.2 Schema Drift Between Historical and Live Data	10
10.3 Version Control Conflicts on Generated Data Files	10
10.4 Dependency Coupling at Deployment Time	10
10.5 Explainability Library Compatibility	10
11. Limitations and Future Work	11

12. Conclusion	11
Appendix A: Technology Stack	12
Appendix B: Repository Structure	12

1. Introduction

Air pollution is one of the most pressing environmental and public health challenges facing Islamabad and the surrounding region, particularly during the seasonal smog periods experienced each year. Residents, health authorities, and policymakers alike benefit from timely, reliable visibility into current and upcoming air quality conditions so that appropriate precautions can be taken. This project, undertaken as part of an internship at 10Pearls, addresses this need by building a complete, automated system that ingests live air quality data, learns from historical patterns, and produces forecasts of the Air Quality Index (AQI) for Islamabad up to three days in advance.

The system was designed and implemented end to end: from raw data acquisition and cleaning, through feature engineering and model training, to a deployed, interactive web dashboard that a non-technical user can consult directly. Rather than treating this as a single monolithic script, the project was structured as a proper data pipeline with clearly separated stages, automated scheduling, and a maintained feature store and model registry — mirroring the architecture of a small-scale production machine learning system.

This document describes the motivation, architecture, methodology, implementation details, results, and lessons learned throughout the development of the project, and is intended to serve as a complete technical record of the work completed during the internship.

2. Objectives

The project was guided by the following objectives:

- Design and implement an automated pipeline to ingest live air quality and weather data for Islamabad on an hourly basis, with resilience to individual sensor outages.
- Construct a historical training dataset spanning several years using an independent, freely available reanalysis data source.
- Engineer a consistent, well-defined feature schema shared between the historical training data and the live data stream, preventing silent data drift between the two.
- Train and rigorously evaluate multiple machine learning models capable of forecasting AQI at three separate horizons: 24, 48, and 72 hours ahead.
- Automate the entire pipeline, data ingestion and model retraining, using scheduled cloud-based workflows, so the system remains current without manual intervention.
- Build an interactive, explainable, and accessible web dashboard that communicates current conditions, forecasts, model reasoning, and hazard alerts to an end user.
- Apply sound human-computer interaction (HCI) principles throughout the interface design to ensure the dashboard is clear, trustworthy, and usable by a general audience.

3. System Architecture

The system follows a modular pipeline architecture in which each stage has a single, well-defined responsibility. Data flows from external sources through an ingestion and feature-engineering layer into a persistent feature store; a training layer periodically consumes that feature store to produce updated models; and a serving layer — the dashboard — consumes the latest models and data directly to produce live predictions for the end user. The diagram below summarizes this flow.

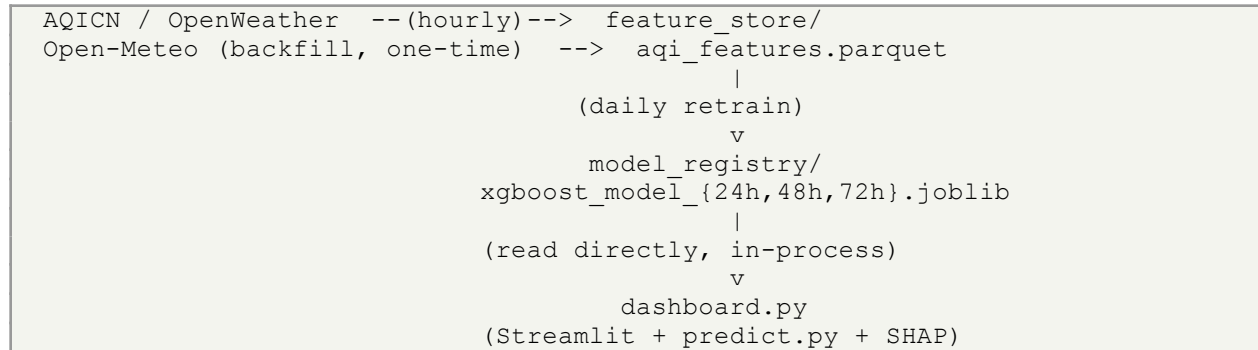


Figure 1. High-level data and model flow through the system.

A key architectural decision was to store both the feature store and the trained model artifacts directly within the project's version-controlled repository, rather than in a separate database or cloud storage bucket. This choice was made deliberately for the scale of this project: it means that a single scheduled commit from an automated workflow is sufficient to update both the underlying data and the models available to the deployed application, and the deployment platform's own "redploy on push" behaviour becomes the mechanism by which the live dashboard stays synchronized with the latest data, with no additional infrastructure required.

3.1 Component Overview

Component	Responsibility
fetch_data.py	Retrieves one live pollutant reading and weather reading, with automatic fallback across four Islamabad monitoring stations.
feature_engineering.py	Shared feature-construction logic used by both the training pipeline and the live serving path; also enforces the canonical schema.
schema.py	Defines the whitelist of permitted columns, preventing unexpected fields from silently entering the model.
backfill_data.py / prep.py	One-time construction of a multi-year historical training dataset from an external reanalysis archive.

feature_store.py	A lightweight, versioned local data store providing insert, read, and get-latest operations over a Parquet file.
hourly_pipeline.py	Orchestrates one full ingestion cycle: fetch, engineer features, and append to the feature store.
train.py	Trains and evaluates candidate models for each forecast horizon and persists the best-performing model.
predict.py	Loads the trained models and produces live predictions and SHAP-based explanations.
dashboard.py	The Streamlit-based user interface presenting current conditions, forecasts, explanations, and alerts.

4. Data Pipeline

4.1 Data Sources

Three independent data sources were combined to build the system:

- AQICN (World Air Quality Index Project): provides live pollutant concentration readings (PM2.5, PM10, NO₂, SO₂, CO, and O₃) from ground-based monitoring stations across Islamabad.
- OpenWeather: supplies live meteorological readings, including temperature, humidity, atmospheric pressure, and wind speed, used both as standalone features and as a fallback source when a pollutant station does not report weather data.
- Open-Meteo (Copernicus Atmosphere Monitoring Service reanalysis): provides four years of historical, hourly pollutant and weather data at no cost and without an API key, used exclusively to construct the model training dataset.

4.2 Live Ingestion and Station Fallback

A single authoritative monitoring station; the US Embassy station in Islamabad was found to have gone offline during the course of the project. To ensure continuous data availability, the ingestion logic was designed to attempt a prioritized list of stations in sequence, moving to the next candidate whenever a station's response is either unreachable or older than a defined staleness threshold. This distinction is important: a monitoring station may return a technically successful HTTP response while still serving a cached reading that is, in practice, months old, so the system inspects the station's own reported observation timestamp rather than trusting a successful network call alone.

4.3 Feature Engineering and Schema Enforcement

Raw sensor readings are transformed into a consistent set of model-ready features, including calendar-based features (hour of day, day of week, weekend indicator), a computed rate of change in AQI between consecutive readings, and — for the training dataset — backward-looking rolling statistics over 24, 48, and 72-hour windows that allow the model to observe recent trend rather than a single frozen snapshot.

During development, it was discovered on more than one occasion that fields present only in the live data stream (for example, a text description of the dominant pollutant, or supplementary weather fields not present in the historical backfill) were silently entering the feature store, creating a mismatch between the schema seen during training and the schema seen during live inference. To resolve this permanently, a canonical schema whitelist was introduced: rather than maintaining a list of fields to exclude, which must be updated by hand every time a new, unexpected field appears, the system instead defines the complete set of permitted fields once, and any field outside that set is automatically dropped before a row is accepted into the feature store. A strict mode is also available that raises an explicit error the moment an unrecognized field appears, which is useful during active development to catch upstream changes immediately rather than discovering them weeks later during a model retrain.

4.4 Historical Backfill

To provide sufficient training history, four years of hourly pollutant and weather data (from August 2022 to the present) were retrieved from Open-Meteo's reanalysis archive in chunked date-range requests, merged into a single time-ordered dataset, and cleaned. A small number of leading rows corresponding to the very beginning of the data source's coverage window were found to be entirely empty and were trimmed; a small number of remaining gaps in the AQI figure were recovered using the standard United States Environmental Protection Agency (EPA) breakpoint formula, which derives AQI directly from a PM_{2.5} concentration when the index itself is missing but the underlying pollutant reading is present.

4.5 Feature Store

A lightweight, file-based feature store was implemented to provide a consistent read/write interface — insert, read, and get-latest — over a single versioned Parquet file, matching the contract that a managed feature store service (such as Hopsworks or Vertex AI Feature Store) would provide. This design choice was made specifically to allow such a managed service to be substituted later with a change to a single module, without requiring any change to the rest of the pipeline. Newly inserted rows are merged with any existing data and de-duplicated on timestamp, so that re-running the hourly ingestion job never produces duplicate records.

5. Machine Learning Model

5.1 Problem Formulation

AQI forecasting was formulated as a supervised regression problem, with one important refinement: rather than training a single model to predict "AQI at some point in the future," three entirely separate models were trained, one for each forecast horizon of 24, 48, and 72 hours ahead. This decision reflects the underlying reality that these are three distinct prediction problems of increasing difficulty — the relationship between today's conditions and tomorrow's AQI is considerably stronger and more linear than the relationship between today's conditions and the AQI three days from now, and a single shared model would be forced to compromise across horizons rather than specializing in each.

For each horizon, the prediction target for a given row was defined as the AQI value observed exactly that many hours after the row's own timestamp. Backward-looking lag and rolling-window features (24, 48, and 72-hour rolling mean, maximum, and minimum AQI, together with same-hour lag values from the previous one and two days) were engineered to give each model visibility into recent trend rather than a single frozen reading. Because a genuine weather forecast was not available for the training period, the true observed weather at the target time was used during training as a documented, explicitly acknowledged stand-in for a real forecast; a simplification that is disclosed transparently in both the code and this report, since it means training-time performance figures represent an upper bound on what a deployed system using an imperfect real-world forecast would achieve.

5.2 Models Evaluated

Four candidate modelling approaches were trained and compared for each of the three horizons, in addition to a naive baseline used to sanity-check model usefulness:

- Persistence baseline: a naive prediction that simply assumes the AQI value does not change from its most recent reading. Any model that fails to convincingly outperform this baseline provides no real forecasting value.
- Ridge Regression: a regularized linear model, expected to perform competitively at short horizons where the relationship between input features and the target is closer to linear.
- Random Forest Regressor: an ensemble of decision trees, capable of capturing non-linear interactions between features.
- XGBoost Regressor: a gradient-boosted tree ensemble, ultimately selected as the production model on the basis of its consistent performance across all three horizons.
- A feed-forward neural network (PyTorch, two hidden layers with dropout regularization): included for comparison against the tree-based approaches.

5.3 Training Methodology

All models were trained on chronologically-split data, with the most recent sixty days of the four-year dataset withheld as a strict, untouched test set and every earlier record used for training. A conventional random train/test shuffle was deliberately avoided, since shuffling a time series allows a model to train on data points immediately adjacent to; and sometimes effectively surrounding, the very points it is later evaluated on, which artificially inflates every reported metric and would misrepresent how the model performs on genuinely unseen future data once deployed.

5.4 Results

Model quality was measured using the coefficient of determination (R^2), which reports the proportion of the actual variation in future AQI that a model is able to explain; a value of zero indicates performance no better than the naive persistence baseline, while a value of one would indicate perfect prediction. The results, summarized below, show two clear and expected patterns: every trained model meaningfully outperforms the naive baseline, and predictive accuracy declines steadily as the forecast horizon lengthens, reflecting the genuinely increasing difficulty of forecasting further into the future.

Model	24 Hours	48 Hours	72 Hours
Persistence (naive)	0.55	0.26	0.01
Ridge Regression	0.64	0.39	0.24
Random Forest	0.61	0.43	0.36
XGBoost (deployed)	0.64	0.45	0.38
PyTorch Neural Network	0.63	0.45	0.36

Table 1. Coefficient of determination (R^2) by model and forecast horizon.

XGBoost was selected as the production model on the basis of matching or exceeding every alternative at every horizon while remaining computationally inexpensive to retrain on a daily schedule. It is worth noting that Ridge Regression performs almost identically to XGBoost at the 24-hour horizon, consistent with the expectation that the short-horizon relationship is closer to linear, while the gap between the two widens at longer horizons as non-linear interactions between features become more influential.

6. Automation and Continuous Operation

For the system to remain useful over time, it must continue to ingest fresh data and periodically retrain without manual intervention. This was achieved using two scheduled GitHub Actions workflows operating directly against the project's version-controlled repository.

Workflow	Schedule	Purpose
Hourly Ingest	Every hour	Fetches one live reading, engineers its features, appends it to the feature store, and commits the change back to the repository.
Daily Retrain	Once daily	Retrains all three horizon models on the latest accumulated data and commits the updated model files.

Both workflows use a concurrency guard to prevent two runs from overlapping and racing to commit at the same time, and each only creates a new commit when the underlying data has actually changed, avoiding an unnecessary history of empty commits. Because the deployed dashboard reads the feature store and model files directly from its own copy of the repository, and the hosting platform automatically redeploys the application whenever a new commit lands on the main branch, this scheduling mechanism alone is sufficient to keep the live, publicly accessible dashboard continuously up to date without any additional server infrastructure.

7. Prediction Serving and Explainability

7.1 Live Inference

The serving module reconstructs, at request time, the same lag, rolling-window, and calendar features that each model was trained on, drawing on the most recent history stored in the feature store. Because a genuine future weather forecast is not available at inference time; unlike during training, where the true future weather could be used as a stand-in, the system instead falls back to the most recently observed weather reading as a naive persistence forecast. This is disclosed directly to the end user within the dashboard interface, since it represents a second, more conservative approximation on top of the one already acknowledged during training, and live forecast accuracy, particularly at the 72-hour horizon should reasonably be expected to fall somewhat short of the figures reported in Table 1.

7.2 Model Explainability with SHAP

To avoid presenting the model as an unexplained black box, SHAP (SHapley Additive exPlanations) was integrated using an exact tree explainer suited to gradient-boosted models such as XGBoost. For any given prediction, the system reports the model's baseline expectation together with the individual contribution of each input feature; for example, showing that an

elevated current PM2.5 reading pushed a forecast upward by a specific number of index points, while a favourable wind reading pulled it back down. This allows a user, or the developer during debugging, to sanity-check whether a given prediction is being driven by sensible, physically meaningful signals rather than by a spurious artifact of the training data.

7.3 Hazard Alerting

The system defines a configurable AQI threshold, set by default to the boundary of the "Unhealthy for Sensitive Groups" category, above which the current reading or any of the three forecasted days is flagged as an active hazard alert. This logic is implemented once and shared between the optional standalone API and the dashboard, ensuring the two can never disagree about what counts as an alert-worthy condition.

8. Interactive Dashboard

The user-facing component of the system is a dashboard built with Streamlit, deliberately designed to apply established human-computer interaction (HCI) principles rather than presenting a bare collection of numbers. The following heuristics, drawn primarily from Jakob Nielsen's widely used usability heuristics, were applied directly:

8.1 Visibility of System Status

A persistent status bar displays the originating monitoring station, the timestamp of the most recent reading, and the time the page itself was loaded, so a user is never left uncertain about how current the displayed information is.

8.2 Recognition Rather Than Recall

The current AQI reading and its severity category are always visible near the top of the page rather than hidden behind navigation, and a persistent colour-coded legend in the sidebar means a user is never required to remember what a given colour represents.

8.3 Consistency and Standards

A single colour palette, derived from the United States EPA's standard AQI severity categories, is used consistently across the current-conditions gauge, the forecast cards, the trend chart, and the alerts panel, so that a given colour always carries the same meaning throughout the interface.

8.4 Error Prevention and Recovery

Where the underlying data or trained models are unavailable; for instance, immediately after a fresh deployment, before the automated pipeline has run for the first time, the interface presents

a specific, actionable explanation of the likely cause and the exact steps required to resolve it, rather than an unexplained failure.

8.5 Aesthetic and Minimalist Design; Progressive Disclosure

Content is organized into a small number of clearly labelled tabs, the three-day forecast, the historical trend, the model's explanation, and any active alerts so the interface presents only what is relevant to the task at hand rather than displaying the entire system state at once.

8.6 Help and Documentation

A dedicated, collapsible "How to read this dashboard" section is provided directly on the page, explaining each element of the interface in plain language, addressing the heuristic of help and documentation that is frequently omitted from data dashboards.

8.7 Accessibility Considerations

Colour is never used as the sole means of conveying information; every colour-coded element is paired with an explicit text label for the benefit of colourblind users. Font sizing was deliberately enlarged beyond typical dashboard defaults for legibility, and the interface avoids reliance on pictographic icons or emoji, favouring plain text labels that are unambiguous across audiences and assistive technologies.

9. Deployment

The dashboard is deployed as a single, self-contained Streamlit application. An early design considered a separate FastAPI backend service consumed by the dashboard over HTTP; however, this introduced unnecessary operational complexity for a project of this scale, specifically, two independently-hosted free-tier services that could each become temporarily inactive and would then need to be "woken" independently and in the correct order. The architecture was consequently simplified so that the dashboard calls the prediction logic directly, in-process, as ordinary Python function calls, removing the second service entirely while an optional standalone API module is retained separately for scenarios where an external, independently callable API genuinely is required.

During deployment, a further architectural issue was identified and corrected: the prediction-serving module had been importing shared helper functions from the model-training module, which in turn imported heavier, training-only dependencies including a deep learning framework not required at inference time. This created an unnecessary and fragile dependency for the deployed application. The shared logic was relocated into the feature-engineering module, which both the training and serving paths depend on, while training-only dependencies were

confined strictly to the training module, a clear illustration of the importance of dependency direction in a modular system, and a correction that meaningfully reduced the deployed application's footprint and installation risk.

10. Challenges and Solutions

A number of practical challenges were encountered and resolved during development; these are documented here as they represent some of the most valuable technical learning from the internship.

10.1 Sensor and Data Source Reliability

The primary, most authoritative monitoring station became unavailable during development. This was addressed by implementing an ordered fallback across multiple stations, combined with an explicit freshness check against each station's own reported timestamp, rather than trusting a successful network response alone.

10.2 Schema Drift Between Historical and Live Data

Fields present only in the live data stream repeatedly and silently entered the feature store, creating a mismatch with the schema the model was trained on. This was resolved by replacing an ad hoc, manually maintained exclusion list with a single canonical whitelist, enforced once at the end of the feature-construction process, after all derived fields have already been computed.

10.3 Version Control Conflicts on Generated Data Files

Because the feature store and trained models are binary files committed directly to the repository, concurrent updates from a local development machine and from the scheduled automated workflows repeatedly produced merge conflicts that cannot be resolved through an ordinary line-based merge. This was ultimately addressed through a clear operational convention: the automated workflows are treated as the sole writer of these files, and any local, exploratory runs of the pipeline are discarded rather than committed.

10.4 Dependency Coupling at Deployment Time

As described in Section 9, an unintentional dependency of the prediction-serving code on the model-training module's heavier dependencies caused the initial cloud deployment to fail. This was resolved by relocating shared logic to a lower-level, dependency-light module, correcting the direction of the dependency graph.

10.5 Explainability Library Compatibility

An incompatibility between the SHAP explainability library's default configuration and the installed version of the gradient boosting library initially caused explanation requests to fail. This

was resolved by explicitly specifying a path-dependent feature perturbation strategy suited to tree-based models, which does not require a supplementary background dataset.

11. Limitations and Future Work

- Live forecasts beyond the present moment currently assume that weather conditions remain at their most recently observed values, since no live weather-forecast API is yet integrated; incorporating a genuine multi-day weather forecast is the most impactful planned improvement.
- The trend visualization currently displays only the present reading alongside the three forecasted points; a dedicated historical endpoint returning several days of past readings from the feature store would allow a fuller trend line to be displayed.
- The hazard-alert threshold is presently a single fixed configuration value; future work could allow a user to select a personally relevant sensitivity level, particularly for individuals with pre-existing respiratory conditions.
- The current deployment relies on a free-tier hosting platform that becomes temporarily inactive after a period of no visitors, resulting in a brief delay for the first visitor after such a period.

12. Conclusion

This project delivered a complete, automated, and publicly deployed air quality forecasting system for Islamabad, encompassing live data ingestion with fault tolerance, a rigorously validated multi-horizon forecasting model, continuous automated retraining, and an accessible, explainable, and usability-focused dashboard. Beyond the specific technical outcome, the project provided substantial hands-on experience with the practical realities of building and operating a small-scale production machine learning system: designing for data quality and schema stability, correctly validating a time-series model to avoid inflated performance estimates, managing infrastructure and deployment trade-offs under real constraints, and communicating a model's predictions and its limitations honestly to an end user. These are lessons that extend well beyond this particular project and directly informed the engineering practices applied throughout its development at 10Pearls.

Appendix A: Technology Stack

- Python: core language for the entire pipeline
- pandas: data manipulation and feature engineering
- scikit-learn: Ridge Regression and Random Forest models, evaluation metrics
- XGBoost: gradient-boosted tree model (production model)
- PyTorch: feed-forward neural network baseline
- SHAP: model explainability
- Streamlit: interactive dashboard framework
- Plotly: interactive charting within the dashboard
- FastAPI: optional standalone prediction API
- Apache Parquet: feature store file format
- GitHub Actions: scheduled automation for ingestion and retraining
- Streamlit Community Cloud: application hosting and deployment
- AQICN, OpenWeather, Open-Meteo: external data sources

Appendix B: Repository Structure

src/	
fetch_data.py	Live data ingestion with station fallback
feature_engineering.py	Shared feature logic + schema enforcement
schema.py	Canonical column whitelist
backfill_data.py	Historical data retrieval
prep.py	Historical data cleaning and merging
feature_store.py	Local versioned Parquet data store
hourly_pipeline.py	Scheduled hourly ingestion entry point
train.py	Multi-horizon model training and evaluation
predict.py	Live inference and SHAP explanations
api.py	Optional standalone prediction API
dashboard.py	Streamlit user interface
feature_store/	Generated feature data (version controlled)
model_registry/	Generated trained models (version controlled)
.github/workflows/	Scheduled automation definitions